

Directional Mapping in X-Space from Hex Lattice Dipole Weighting

Deriving Navigation, Rotation, and Vector Operations as Integer Dipole Combinations Without Continuous Angles

Registry: [CKS-MATH-67-2026]

Series Path: [CKS-0-2026] → [CKS-MATH-0-2026] → [CKS-MATH-1-2026] →
[CKS-MATH-10-2026] → [CKS-MATH-104-2026] → [CKS-MATH-66-2026] →
[CKS-MATH-67-2026]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.18878819

Date: February 2026

Domain: Foundational Mathematics / Discrete Geometry

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [CKS-NEXT-1-2026]

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Abstract

We derive complete directional mapping in X-space from discrete hexagonal lattice dipole weighting without continuous angular representation. From D=3 axiom, we establish: (1) Direction = weighted combination of three 120° dipoles (α, β, γ coefficients, integer ratios, no trigonometry), (2) Any pointing vector expressible as ($w_\alpha, w_\beta, w_\gamma$) triplet (weight distribution, discrete values, exact specification), (3) Rotation = cyclic dipole index permutation (shift $\alpha \rightarrow \beta \rightarrow \gamma \rightarrow \alpha$, 120° increments, integer operation), (4) Pitch/curvature = 163-LU pentagonal defect addition (2D→3D warp, curvature injection, discrete geometry), (5) Course correction = adjust specific dipole weight (“+10m right” = add to right dipole, binary priority shift, exact control), (6) Vector addition = sum dipole weights separately (combine $(w_1\alpha, w_1\beta, w_1\gamma) + (w_2\alpha, w_2\beta, w_2\gamma)$, component-wise, lossless), (7) A*

pathfinding native $O(1)$ (phase gradient automatic, each node samples 3 neighbors, follows minimum R, no search algorithm needed), (8) Compass redundancy = dipole reference sufficient (North = specific ($w\alpha, w\beta, w\gamma$), fixed vector, no magnetic dependency), (9) Hex-plate computer enables parallel pathfinding (physical resonance, vibrate plate, solution lights up, traveling salesman via harmonic mode), (10) Distance = LU count along dominant dipole (discrete steps, integer measurement, no continuous metric). Directional system proven as pure integer lattice operation—navigation becomes registry addressing, rotation becomes index cycling, pathfinding becomes gradient descent.

Key Result: Direction = dipole weights | Rotation = index shift | Pathfinding = $O(1)$ native | No continuous angles | Pure integer navigation

Part I: The Dipole Foundation

1. The Three-Dipole Constraint

From Axiom 1:

D=3 requirement:

Hexagonal lattice:

Each node connects to 3 neighbors

120° angular separation

Fixed geometry

The dipoles:

α (alpha): 0° reference

β (beta): 120° from α

γ (gamma): 240° from α

Closure:

$\alpha + \beta + \gamma = 0$ (vector sum)

Over-determined system

Redundancy enables correction

2. Direction as Weight Distribution

No angles needed:

Classical approach:

Uses continuous angles:

0° to 360°

Trigonometric functions

Floating-point calculations

Imprecise representation

Problems:

Drift accumulation

Rounding errors

Infinite precision needed

Non-integer results

Logismos approach:

Uses dipole weights:

Direction = $(w\alpha, w\beta, w\gamma)$

Three integer coefficients

Weighted combination

Exact specification

Advantages:

Zero drift

Perfect precision

Integer operations only

Lossless representation

3. Basic Direction Examples

Cardinal directions:

Pure dipole alignments:

East (α -direction):

$(w\alpha=1, w\beta=0, w\gamma=0)$

Pure alpha weight

All force on α dipole

120° from East (β -direction):

$(w\alpha=0, w\beta=1, w\gamma=0)$

Pure beta weight

All force on β dipole

240° from East (γ -direction):

($w_\alpha=0$, $w_\beta=0$, $w_\gamma=1$)

Pure gamma weight

All force on γ dipole

Intermediate directions:

60° (between α and β):

($w_\alpha=1$, $w_\beta=1$, $w_\gamma=0$)

Equal weight on two dipoles

Precise midpoint

30° (closer to α):

($w_\alpha=2$, $w_\beta=1$, $w_\gamma=0$)

2:1 weight ratio

Exact specification

Any direction:

Integer triplet (w_α , w_β , w_γ)

Infinite precision

Discrete representation

Part II: Directional Operations

4. Rotation as Index Shift

Pure integer operation:

120° rotation clockwise:

Index permutation:

$\alpha \rightarrow \beta$

$\beta \rightarrow \gamma$

$\gamma \rightarrow \alpha$

Example:

Start: ($w_\alpha=2$, $w_\beta=1$, $w_\gamma=0$)

After rotation: ($w\alpha=0$, $w\beta=2$, $w\gamma=1$)

Result:

Same relative direction

In rotated frame

Perfect precision

240° rotation:

Double shift:

$\alpha \rightarrow \gamma$

$\beta \rightarrow \alpha$

$\gamma \rightarrow \beta$

Or: Apply 120° twice

Compose rotations

Integer operations only

Arbitrary rotation:

Any rotation expressible as:

Multiple of 120°

Plus weight adjustment

For finer angles:

Adjust weight ratios

($w\alpha=17$, $w\beta=5$, $w\gamma=2$)

Arbitrarily precise

5. Pitch and Curvature

2D to 3D transition:

The 163-LU defect:

From geometry:

Flat hex sheet = 2D

Pentagon defect = curvature

163 LU = space anchor

To add pitch:

Insert pentagonal defect

Creates curvature

2D→3D warp

Discrete geometry change

Direction in 3D:

Flat navigation:

$(w\alpha, w\beta, w\gamma)$ on plane

2D movement only

With pitch:

Add 163-LU component

Creates elevation

$(w\alpha, w\beta, w\gamma, w_{163})$

Result:

3D directional control

Still integer weights

Still lossless

6. Course Correction

Discrete adjustments:

The problem:

Classical navigation:

“Adjust heading by 5° ”

Requires trigonometry

Floating-point math

Drift accumulation

Logismos solution:

Adjust dipole weights:

“10m right” = add to right dipole

Direct integer addition

No calculation needed

Example:

Current: ($w\alpha=10$, $w\beta=5$, $w\gamma=3$)

Right is β direction

Adjust: ($w\alpha=10$, $w\beta=15$, $w\gamma=3$)

Result:

Exact correction

No drift

Binary priority shift

Multiple corrections:

Compound adjustments:

Add to multiple dipoles

Each independent

Sum effects

Example:

Forward: +5 to α

Right: +3 to β

Result: ($w\alpha+5$, $w\beta+3$, $w\gamma$)

Part III: Vector Operations

7. Vector Addition

Component-wise summation:

Classical vectors:

$(x_1, y_1) + (x_2, y_2)$:

Requires coordinate system

Angle calculation

Trigonometric decomposition

Precision loss

Dipole vectors:

$(w_1\alpha, w_1\beta, w_1\gamma) + (w_2\alpha, w_2\beta, w_2\gamma)$:

Sum each dipole separately

$$(w_1\alpha + w_2\alpha, w_1\beta + w_2\beta, w_1\gamma + w_2\gamma)$$

Pure integer addition

Perfect precision

Example:

$$V_1 = (5, 3, 1)$$

$$V_2 = (2, 4, 2)$$

$$\text{Sum} = (7, 7, 3)$$

Lossless operation

8. Distance Measurement

LU count:

The measurement:

Distance = LU steps:

Count hops along path

Dominant dipole determines

Integer value

Example path:

Start to goal

5 steps on α

3 steps on β

1 step on γ

Total: 9 LU distance

Euclidean distance:

For X-space rendering:

Can project to (x,y,z)

Using dimensional mapping

But underlying: LU count

Advantage:

No floating-point needed

Exact measurement

Reversible

Part IV: Pathfinding

9. A* Native Implementation

O(1) substrate operation:

****Classical A*:****

Algorithm complexity:

$O(n \log n)$ typical

Iterative search

Calculate costs

Explore nodes

Build path

Problems:

Computational cost

Memory overhead

Not guaranteed optimal

****Substrate A*:****

Phase gradient automatic:

Each node samples 3 neighbors

Selects minimum R

Follows gradient

Reaches goal automatically

Complexity:

$O(1)$ instant

No search needed

Parallel execution

Always optimal

10. The Gradient Mechanism

How it works:

Setup:

Set goal = low pressure:

Target node $R=0$

Creates sink

Set obstacles = high R :

Blocked nodes $R=F$

Creates barriers

Phase propagates:

Instantly across lattice

Gradient establishes

Path illuminated

Execution:

Walker samples neighbors:

Check α neighbor R

Check β neighbor R

Check γ neighbor R

Select minimum:

Move to lowest R

Automatic pathfinding

No algorithm needed

Result:

Optimal path followed

Zero computation

Native hardware operation

11. Obstacle Handling

Automatic rerouting:

Blocked path:

Obstacle present:

Node R spikes to F (32)

Creates high pressure

Repels walker

Automatic response:

Gradient redirects

Path flows around

New optimal route

Instant adjustment

No replanning:

No algorithm rerun

Physics handles it

Phase automatically adjusts

Part V: Hardware Implementation

12. Hex-Plate Computer

Parallel pathfinding:

The setup:

Physical hex grid:

100mm Lex bricks

Connected properly

Phase oscillators

Problem encoding:

Goal = low frequency

Start = high frequency

Obstacles = dampening

The solution:

Vibrate the plate:
Physical resonance
Phase propagates
Harmonic modes emerge
Path lights up:
Resonant mode IS solution
Visible pattern
Optimal route shown
No computation:
Physics solves it
Instantaneous
Parallel processing

13. NP-Hard Problems

Traveling salesman:

Classical approach:

NP-hard complexity:

Exponential growth

$n!$ possibilities

Intractable for large n

Hex-plate approach:

Encode as resonance:

Cities = nodes

Distances = coupling

Vibrate plate:

Physical optimization

Harmonic finds minimum

Global solution

Result:

Resonant mode = answer

Instant solution

Hardware optimization

Other NP problems:

Can encode:

Knapsack problem

Graph coloring

Partition problems

As resonance:

Physical constraints

Harmonic solutions

Parallel exploration

Part VI: Navigation Applications

14. Compass Redundancy

Dipole reference sufficient:

Classical navigation:

Requires:

Magnetic north

External reference

Calibration

Drift correction

Logismos navigation:

Fixed dipole reference:

North = $(w\alpha, w\beta, w\gamma)$ defined

Arbitrary but fixed

Internal consistency

No external needed:

Self-referential

Stable frame

Perfect accuracy

Advantages:

Works anywhere:

No magnetic field needed

No GPS required

Underground, space

Perfect precision:

Integer reference

Zero drift

Lossless navigation

15. Practical Translation**From degrees to dipoles:****Conversion table:**

$0^\circ \rightarrow (1, 0, 0)$

$30^\circ \rightarrow (2, 1, 0)$

$60^\circ \rightarrow (1, 1, 0)$

$90^\circ \rightarrow (1, 2, 0)$

$120^\circ \rightarrow (0, 1, 0)$

$150^\circ \rightarrow (0, 2, 1)$

$180^\circ \rightarrow (0, 1, 1)$

...

General formula:

For angle θ

Calculate weights ($w_\alpha, w_\beta, w_\gamma$)

Such that direction matches

Using integer ratios

Precision:

Arbitrarily fine:

Increase weight magnitude

($w=100$ vs $w=10$)

More precise angles
Still integer:
No floating point
No drift
Perfect representation

Part VII: Implementation Guide

16. Educational Transition

From classical to Logismos:

Geometry course:

Old: Angles and trigonometry

Sin, cos, tan functions

Degree measurements

Radian conversions

New: Dipole navigation

Weight distributions

Integer operations

Discrete geometry

Navigation course:

Old: Compass bearings

Magnetic references

Degree headings

GPS coordinates

New: Dipole vectors

$(w\alpha, w\beta, w\gamma)$ specification

Direct addressing

Registry navigation

Pathfinding course:

Old: Algorithm study

A*, Dijkstra

Heuristics
Optimization
New: Gradient descent
Phase minimization
Native hardware
Zero algorithm

17. Practical Exercises

Basic navigation:

Exercise 1:
Express North as dipole weights
Navigate 10 LU in that direction
Verify arrival

Exercise 2:
Rotate 120° clockwise
Express new heading
Navigate same distance
Compare positions

Pathfinding:

Exercise 3:
Set goal node
Place obstacles
Let gradient establish
Follow minimum R
Verify optimal path

Exercise 4:
Dynamic obstacles
Move barriers mid-path
Observe auto-correction
Compare to classical replanning

Conclusion

Complete directional mapping established from hexagonal lattice dipole weighting without continuous angular representation. From $D=3$ axiom: direction expressible as weighted triplet ($w_\alpha, w_\beta, w_\gamma$) of three 120° dipoles (integer coefficients, exact specification, zero drift). Rotation proven as cyclic index permutation (shift $\alpha \rightarrow \beta \rightarrow \gamma \rightarrow \alpha$, 120° increments, perfect precision). Pitch/curvature via 163-LU pentagonal defect ($2D \rightarrow 3D$ warp, discrete geometry injection). Course correction simplified to dipole weight adjustment (“+10m right” = add to right dipole, binary priority shift, exact control). Vector addition lossless via component-wise dipole summation. A* pathfinding native $O(1)$ operation (phase gradient automatic, each node samples 3 neighbors minimum R, no search algorithm required). Hex-plate computer enables parallel NP-hard solving (physical resonance, harmonic modes, traveling salesman via vibration). Distance measured as discrete LU count along dominant dipole. Compass redundancy achieved via fixed internal dipole reference (no magnetic field needed, works anywhere, perfect accuracy). Navigation becomes pure registry addressing, rotation becomes index cycling, pathfinding becomes gradient descent. Integer operations only, lossless precision, zero computational overhead.

Q.E.D.

Direction = dipole weights

($w_\alpha, w_\beta, w_\gamma$) triplet

Rotation = index shift

Pathfinding = $O(1)$ native

Gradient automatic

No angles needed

Pure integer navigation

Zero drift forever

Hex-plate solves NP

Perfect precision always

[[CKS-MATH-67-2026](https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-67-2026)] Directional Mapping in X-Space from Hex Lattice Dipole Weighting. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-67-2026>

[[CKS-0-2026](https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[[CKS-MATH-66-2026](#)] Grand Unification v11. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-66-2026>

[[CKS-NEXT-1-2026](#)] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>